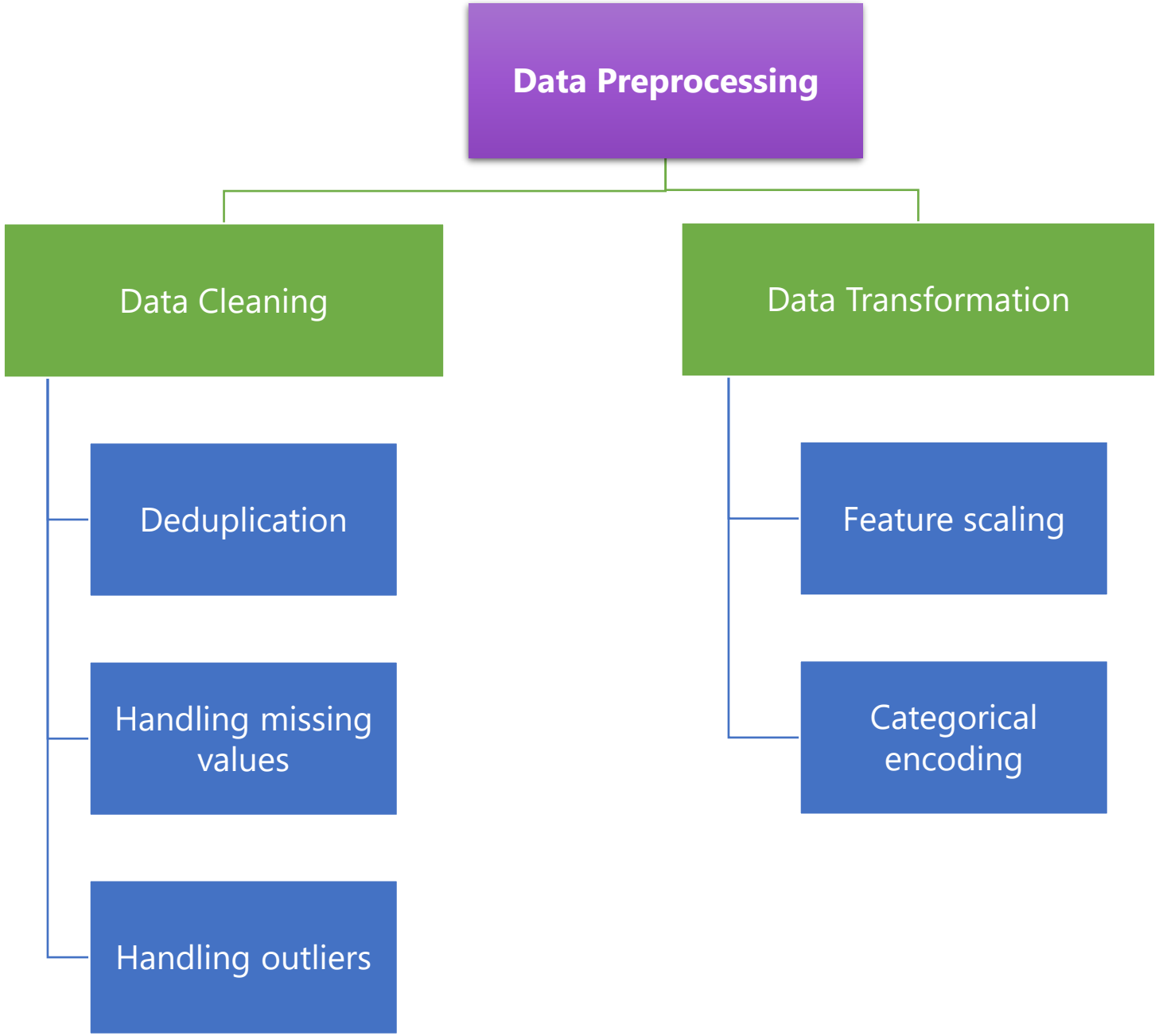
Data Preparing for Machine Learning



Dr. Abdelkarim Erradi
CSE@QU

With assistance from AI tools, particularly Microsoft Copilot and Google Gemini

Outline


Data Preprocessing
Data Cleaning
Data Transformation
Deduplication
Handling missing values
Handling outliers
Feature scaling
Categorical encoding

Data Deduplication





05.dp**01-deduplication**

Data Deduplication

- **Duplicates:** identical rows describing the same real-world entity or event
- **Why they matter**
  - Repeated rows get extra weight and **bias the model**
  - Copies in train and test sets **inflate accuracy**
- **Where they come from**
  - Same person submits a form twice
  - Retweets: same content, new metadata
- **How to detect them**
  - **Exact matching:** compare rows directly
  - **Near-duplicates:** similarity, distance or clustering

Example: orders table

order_id	customer	item	price	Result
101	Sara	Laptop	900	Keep
102	Omar	Mouse	25	Keep
103	Sara	Laptop	900	**Duplicate → drop**
104	Lina	Phone	600	Keep

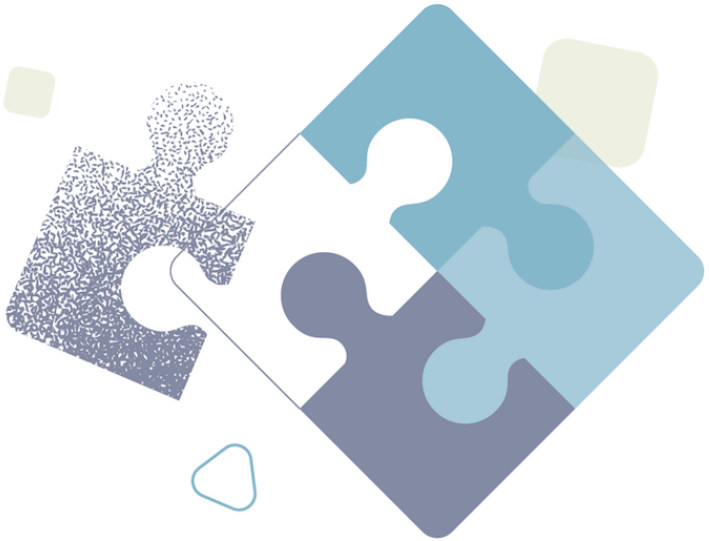
Only order_id differs, so ignore it when comparing.

```python
# Columns that define "the same" record (no ID)
cols = ["customer", "item", "price"]

# 1) Count the duplicates
print(df.duplicated(subset=cols).sum())

# 2) Drop them (keeps the first copy)
df_clean = df.drop_duplicates(subset=cols)
```

Handling Missing Values





05.dp\02-handling-missing-values

Missing Values: The Big Picture

1 Why is it missing?

MCAR

Completely at random

Pure chance, e.g. a sensor fails or a survey page is not completed. Safest case to impute.

MAR

At random (given other columns)

Depends on other features, e.g. income is missing more often for younger people.

MNAR

Not at random

Depends on the value itself, e.g. very high earners skip the income question.

2 How much is missing in a column?

< 5%	Simple imputation (mean / median / mode) is usually safe
5 – 30%	Impute and add a missing indicator; try model-based imputation
30 – 60%	Be careful: heavy imputation distorts the data. Ask a domain expert
> 60%	Consider dropping the column

Step 1: Find the Missing Values

- **Count and percentage** per column with **isna()** shows where the gaps are
- **Hunt for disguised missing values:** "NA", "n/a", "null", "_" or codes like -99 look like data, so pandas will not count them
- **Ask why:** random, related to other columns, or related to the value itself? The answer picks the fix

Python | pandas

```python
import pandas as pd
import numpy as np

# 1. Count and % of missing values per column
report = pd.DataFrame({
    "missing": df.isna().sum(),
    "pct": (df.isna().mean() * 100).round(1),
})
print(report.sort_values("pct", ascending=False))

# 2. Turn "disguised" missing values into real NaN
fake_na = ["NA", "n/a", "None", "null", "_", "",
"empty"]
df = df.replace(fake_na, np.nan)
df["parking_space"] = df["parking_space"].replace(-99,
np.nan)
```

Step 2: Drop What You Can't Fix

- **Drop columns** that are mostly empty (> 60% missing): they add noise and little signal
- **Drop rows** only when a critical value is missing, e.g. the target you want to predict
- **Be careful:** if many rows are affected, dropping throws away too much data. Impute instead

Python | pandas

```python
# Drop columns with more than 60% missing values
too_empty = df.columns[df.isna().mean() > 0.60]
df = df.drop(columns=too_empty)
print("Dropped:", list(too_empty))

# Drop rows that miss a critical value (e.g. the target)
df = df.dropna(subset=["income"])

# Check what is left
print(df.shape)
print(df.isna().sum())
```

Rule of thumb: remove a column only if it is missing > 60% of its values; for less missing data use imputation (next steps).

Step 3: Mean / Median / Mode Imputation

- **Numeric:** mean if roughly symmetric, **median** if skewed or has outliers
- **Categorical: mode** (most frequent value)
- **Use when** data is missing at random and at most ~5% per column
- **Limitation:** it shrinks variance and distorts correlations; more missing means more distortion

Avoid data leakage: fit the imputer on the training set only, then transform the test set.

Python | scikit-learn

```python
from sklearn.impute import SimpleImputer

num_imp = SimpleImputer(strategy="median")       # or "mean"
cat_imp = SimpleImputer(strategy="most_frequent")  # mode

# fit on TRAIN only, then apply to train and test
X_train[num_cols] = num_imp.fit_transform(X_train[num_cols])
X_test[num_cols]  = num_imp.transform(X_test[num_cols])

X_train[cat_cols] = cat_imp.fit_transform(X_train[cat_cols])
X_test[cat_cols]  = cat_imp.transform(X_test[cat_cols])

# Quick pandas version (for exploration only)
df["age"] = df["age"].fillna(df["age"].median())
```

Step 4: When “Missing” Means Something

- **Missing indicator:** a 0/1 column saying “was missing”; the gap itself may predict the target
- **Constant value:** a clearly fake number (-99) for “not applicable”, or a “high” guess when high earners hide income (MNAR)
- **“Missing” category:** one explicit label for all categorical gaps
- **Interpolate:** for ordered or time-series data, use the previous and next values

Python | pandas

```python
# 1. Missing indicator (flag) + dummy value
df["parking_missing"] =
df["parking_space"].isna().astype(int)
df["parking_space"] = df["parking_space"].fillna(-99)

# 2. MNAR: high earners skip income -> fill with a
"high" guess
df["income"] = df["income"].fillna(250)

# 3. Categorical: explicit "Missing" label
df["entree"] = df["entree"].fillna("Missing")

# 4. Ordered data: average of previous and next value
df["years_seniority"] =
df["years_seniority"].interpolate(method="linear")
```

Tip: tree-based models can use the flag + dummy value directly.

Putting It Together: Imputation Pipeline

Python | scikit-learn

```python
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer

num_cols = ["age", "income", "parking_space"]
cat_cols = ["entree", "pets"]

prep = ColumnTransformer([
    ("num", SimpleImputer(strategy="median",
                add_indicator=True), num_cols),
    ("cat", SimpleImputer(strategy="constant",
                fill_value="Missing"), cat_cols),
])

X_train_clean = prep.fit_transform(X_train)# learn on train
X_test_clean  = prep.transform(X_test)    # reuse on test
```

Why a pipeline?

No leakage
the imputer learns only from training data

Repeatable
same steps for train, test and new data

Flags for free
add_indicator=True adds the missing flags

Need smarter?
try KNNImputer or IterativeImputer, but start simple

Summary: Choosing a Strategy

Random (MCAR / MAR)
Missingness is not about the value itself

Numeric	Categorical
Mean or median	**Mode**

Not random (MNAR)
Missingness is about the value itself

Numeric	Categorical
Arbitrary / extreme value	**“Missing” category**

In both cases, add a missingness indicator when the missingness itself may carry information.

1

Inspect first
isna() and disguised values

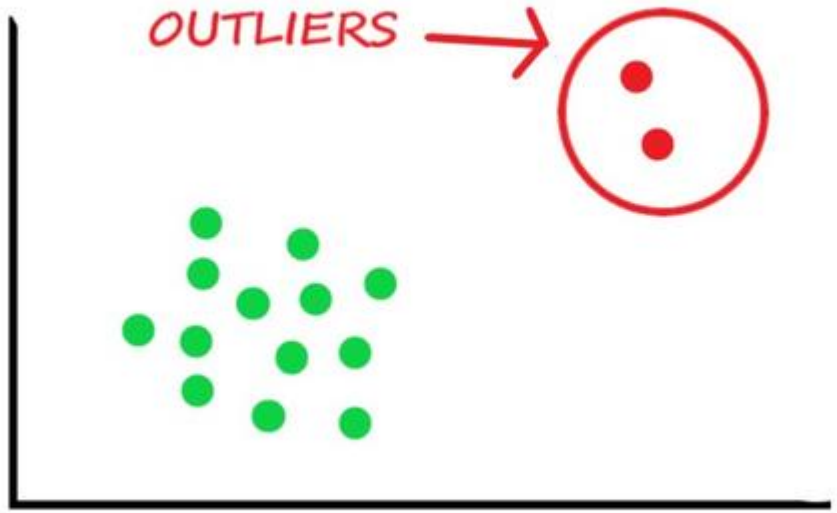
2

Start simple
drop > 60%, impute the rest

3

Fit on train only
avoid data leakage

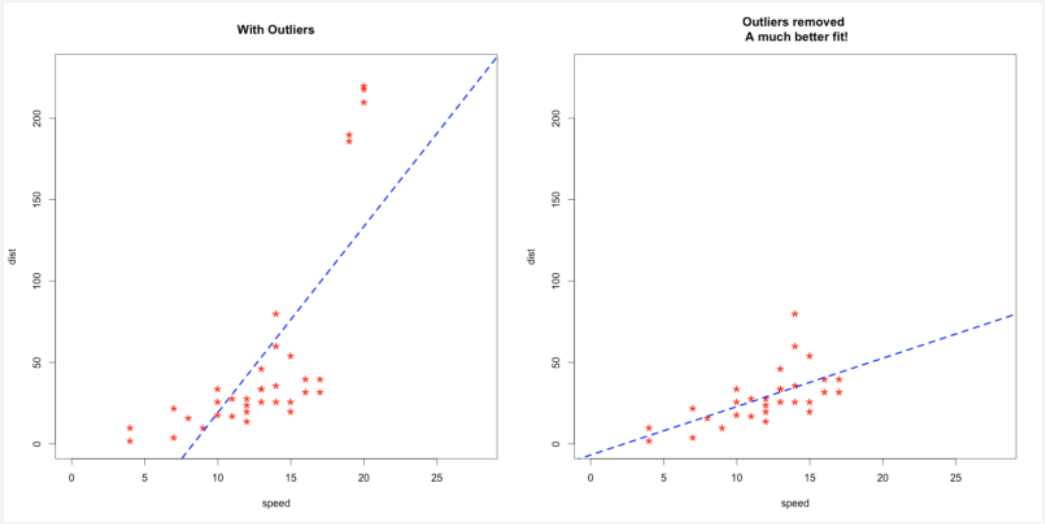
Handling Outliers





05.dp\3-handling-outliers

Step 1: Spot the Outliers



https://www.r-bloggers.com/outlier-detection-and-treatment-with-r/

- **Outlier:** a value far from the rest. It may be a real rare case or an error
- **Sensitive models:** linear / logistic regression and distance-based models. Trees are robust
- **Never delete blindly:** check with domain knowledge first

Python | pandas

```python
import pandas as pd

# 1. Visual check: boxplot and histogram
df["income"].plot.box()
df["income"].plot.hist(bins=30)

# 2. IQR rule (works well for skewed data)
q1, q3 = df["income"].quantile([0.25, 0.75])
iqr = q3 - q1
low  = q1 - 1.5 * iqr
high = q3 + 1.5 * iqr

# 3. Count and inspect the outliers
mask = (df["income"] < low) | (df["income"] > high)
print(mask.sum(), "outliers")
print(df[mask])
```

Tip: the boxplot whiskers use this same IQR rule, so plot first, then count.

Step 2: Choose How to Handle Them

A **Trim (remove)**
Pro: fast and simple. Con: loses rows and information. Use for few, clearly wrong values.

B **Cap (winsorize)**
Pro: keeps every row. Con: slightly distorts the distribution. Most common choice (next slide).

C **Treat as missing**
Pro: no data removed; reuse imputation. Con: may distort the distribution.

D **Bin (discretize)**
Pro: extremes fall into the first or last bin. Con: loses fine detail.

Python | pandas

```python
import numpy as np

# A. Trim: keep only the rows that are NOT outliers
df_trim = df[~mask]

# B. Cap: clip values to the IQR limits (next slide)
df["income_cap"] = df["income"].clip(low,
high)

# C. Treat as missing, then impute with the median
df["income_nan"] = df["income"].mask(mask)
med = df["income_nan"].median()
df["income_nan"] =
df["income_nan"].fillna(med)

# D. Bin: 5 equal-sized groups, extremes land in the
#    first or last bin
df["income_bin"] = pd.qcut(df["income"], q=5,
     labels=False, duplicates="drop")
```

Step 3: Capping (Winsorization)

Replace extreme values with a limit instead of deleting them:

IQR

Skewed data
e.g. income, house prices
Lower = Q1 − 1.5 × IQR
Upper = Q3 + 1.5 × IQR

Z-score

Roughly normal data
e.g. height
Lower = mean − 3σ
Upper = mean + 3σ

Result: below lower limit → lower limit; above upper → upper limit.

Avoid leakage: compute limits on the training set only, then reuse them on the test set.

Python | pandas

```python
# Option 1: IQR rule (skewed data, e.g. income)
q1, q3 = df["income"].quantile([0.25, 0.75])
iqr = q3 - q1
low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr
df["income"] = df["income"].clip(lower=low,
        upper=high)

# Option 2: z-score rule (roughly normal, e.g. height)
m, s = df["height"].mean(), df["height"].std()
df["height"] = df["height"].clip(lower=m - 3 * s,
                                 upper=m + 3 * s)

# Check the effect
print(df[["income", "height"]].describe())
```

Tip: cap only if outliers hurt your model. Compare scores with and without capping.

Feature Scaling

Standardisation	Normalisation
$x_{\text{stand}} = \frac{x - \text{mean}(x)}{\text{standard deviation } (x)}$	$x_{\text{norm}} = \frac{x - \min(x)}{\max(x) - \min(x)}$



05.dp\04-feature-scaling

Feature Magnitude matters

- Features with large ranges **dominate** distance-based models
  - Example: Age spans 20–65, but Salary spans 25,000–26,900
  - Distance used below: |Age difference| + |Salary difference|

Person	Age	Salary
1	20	25,000
2	65	25,500
3	25	25,700
4	22	26,900
5	55	25,400

Before scaling: salary dominates

Person 1 ↔ 2: 45 + 500 = **545**

Person 1 ↔ 3: 5 + 700 = **705** → **Person 2 looks closer**

After min-max scaling (0–1): both features count

Person 1 ↔ 2: 1.00 + 0.26 = **1.26**

Person 1 ↔ 3: 0.11 + 0.37 = **0.48** → **Person 3 is closer**

Takeaway: Persons 1 and 3 are close in age (20 vs 25), yet the raw distance says Person 2 is nearer because salary swamps age. Scale first so every feature gets a fair say.

Feature scaling

- **Feature scaling:** put features measured on different scales onto **a common scale**
  - Usually the last preprocessing step, right before training
  - Does not change the shape of the distribution
- **Why it matters**
  - Large-range features stop dominating the model
  - Distance-based methods work better (KNN, k-means, PCA)
  - Gradient-based training converges faster (neural networks)

Scale these

KNN, k-means, SVM, PCA
Linear / logistic regression
Neural networks

Usually not needed

Decision trees, random forests
Gradient boosting (e.g., XGBoost)

```
# Split first, then scale: learn the scaling numbers from the TRAINING set only
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)   # learn mean/std from train, then apply
X_test_scaled  = scaler.transform(X_test)  # reuse the same numbers (never fit on test!)
```

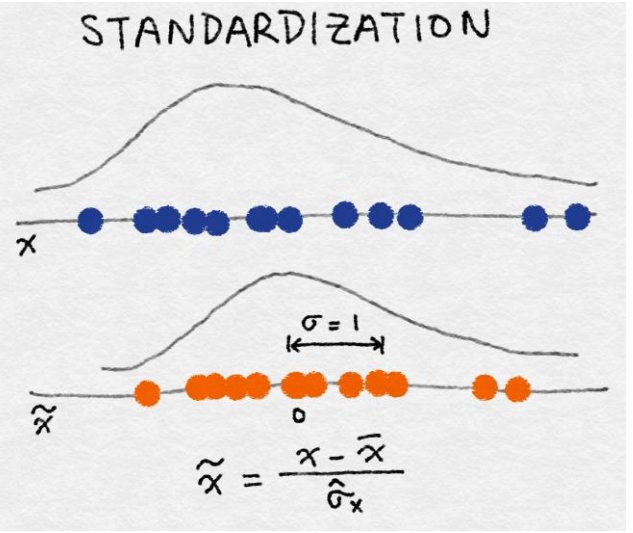
Standardization

- Standardization centers a feature at **zero mean** and scales it to **unit variance**

$$x_i' = \frac{x_i - \mu}{\sigma}$$

x_i' is the standardized feature value, x_i = original value, μ = mean for feature x, and σ = standard deviation

  - Standardized value = number of standard deviations from the mean
  - Less sensitive to outliers than min-max
  - Works best on roughly bell-shaped (normal) data




```python
# Option 1: by hand with pandas
x = df["price"]
# ddof=0 means divide by n, like scikit-learn
df["scaled"] = (x - x.mean()) / x.std(ddof=0)

# Option 2: scikit-learn (same result)
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df["scaled"] = scaler.fit_transform(df[["price"]])
```

Price
100
90
50
40
20
100
50
60
120
40
200

Mean = 79
Standard dev = 51

Obs. - Mean

Standard dev

Price
0.41
0.22
-0.57
-0.76
-1.16
0.41
-0.57
-0.37
0.80
-0.76
2.37

Image source

Min-Max Scaling

- **Rescales a feature to the range 0 to 1**

$$x_i' = \frac{x_i - \min(x)}{\max(x) - \min(x)}$$

x_i' is the scaled value

- Also called **Normalization**
- Sensitive to outliers (they squash the rest)

```
# Option 1: by hand with pandas
x = df["price"]
df["scaled"] = (x - x.min()) / (x.max() - x.min())

# Option 2: scikit-learn (same result)
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df["scaled"] = scaler.fit_transform(df[["price"]])
```

Price
100
90
50
40
20
100
50
60
120
40
200

Max = 200
Min = 20
Range = 200 – 20 = 180



Obs. - Min

Range

Price
0.44
0.39
0.17
0.11
0.00
0.44
0.17
0.22
0.56
0.11
1.00

Feature scaling does not change the shape of the distribution

```python
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler

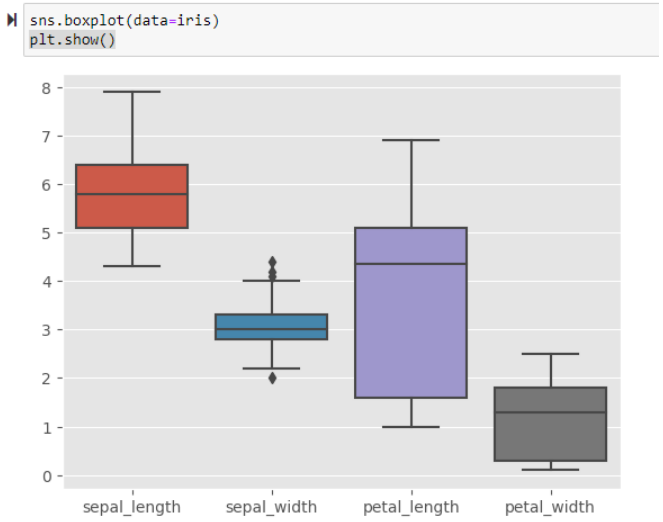
iris = load_iris(as_frame=True).data       # 4 numeric columns
scaled = StandardScaler().fit_transform(iris)
iris_scaled = pd.DataFrame(scaled, columns=iris.columns)
```

What to notice

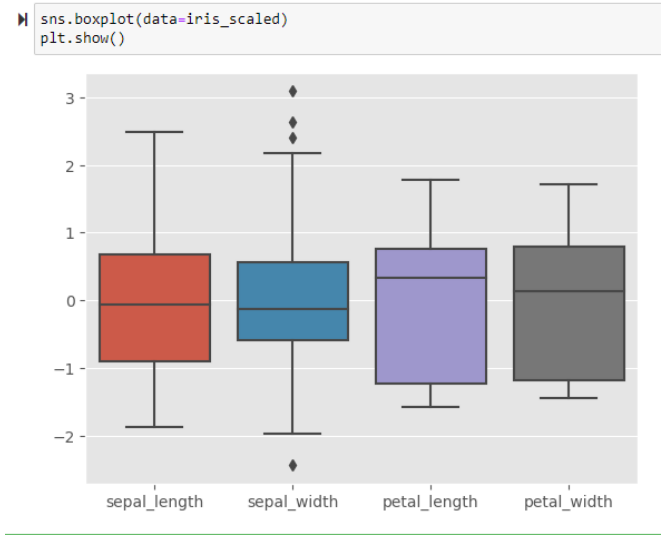
Boxplots: all 4 features now share a similar range

Histograms: same shape, only the x-axis values change

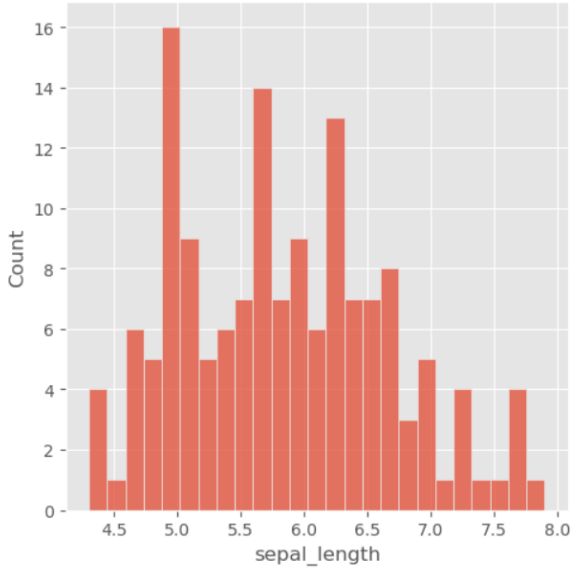
Boxplots: before



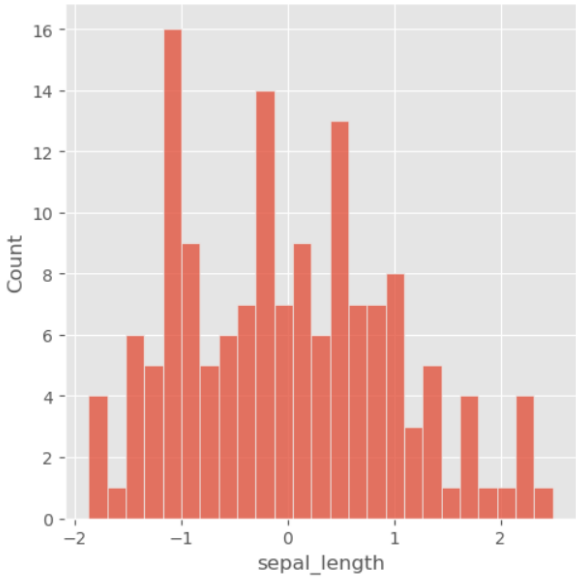
Boxplots: after



sepal_length: before

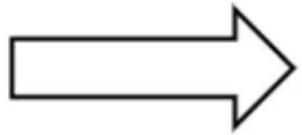


sepal_length: after



Categorical Variable Encoding

Grades
A
B
C
D
Fail



Grades	Encoded
A	4
B	3
C	2
D	1
Fail	0



05.dp**05-categorical-encoding**

Categorical encoding

Models need numbers. Categorical encoding turns text categories into numeric features.

One-hot encoding

Red → [1, 0, 0]

One 0/1 column per category

Best for: linear models, few categories

Ordinal encoding

Small → 0, Medium → 1, Large → 2

One integer per category

Best for: tree models, or ordered categories

Count / frequency

Doha → 3 (seen 3 times)

Category replaced by how often it appears

Best for: many categories, tree models

```python
import pandas as pd

# Small example dataset used on the next slides
df = pd.DataFrame({
    "color": ["Red", "Green", "Blue", "Green", "Red"],
    "size":  ["Small", "Large", "Medium", "Small", "Large"],
    "city":  ["Doha", "Doha", "Rayyan", "Wakra", "Doha"],
})
```

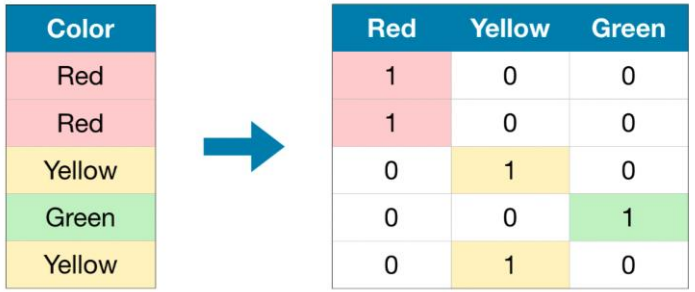
Golden rule

Learn the encoding from the training set only

Then reuse it on the test set, exactly as with scaling.

One hot encoding

- Turns one categorical column into several **0/1 columns**, one per category
  - Value **1** = category present, **0** = absent



Color
Red
Red
Yellow
Green
Yellow

→

Red	Yellow	Green
1	0	0
1	0	0
0	1	0
0	0	1
0	1	0

```python
# Option 1: pandas (quick and simple)
encoded = pd.get_dummies(df["color"], prefix="color", dtype=int)
# drop_first=True removes one redundant column


# Option 2: scikit-learn (for ML pipelines)
from sklearn.preprocessing import OneHotEncoder

enc = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
X_train_enc = enc.fit_transform(X_train[["color"]]) # learn + apply
X_test_enc = enc.transform(X_test[["color"]])       # apply only
```

When to use

Linear models, and any model that assumes no order between categories

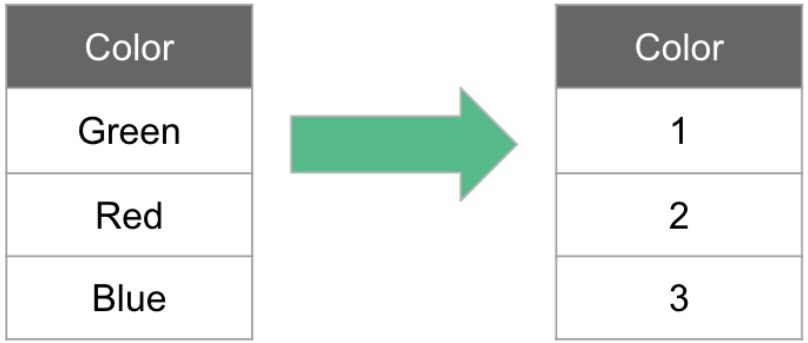
Watch out

Many categories = many columns (high cardinality)

Use **drop_first=True** for linear models to avoid redundant columns

Ordinal Encoding

- Assigns each category a **unique integer** (0 to n-1)
  - Does not expand the feature space
- **When to use:** tree-based models, or categories with a natural order (Small < Medium < Large)




```python
# Option 1: pandas (you choose the order)
size_order = {"Small": 0, "Medium": 1, "Large": 2}
df["size_encoded"] = df["size"].map(size_order)

# Option 2: scikit-learn (give the order explicitly)
from sklearn.preprocessing import OrdinalEncoder

order = [["Small", "Medium", "Large"]]
enc = OrdinalEncoder(categories=order)
df["size_encoded"] = enc.fit_transform(df[["size"]])
```

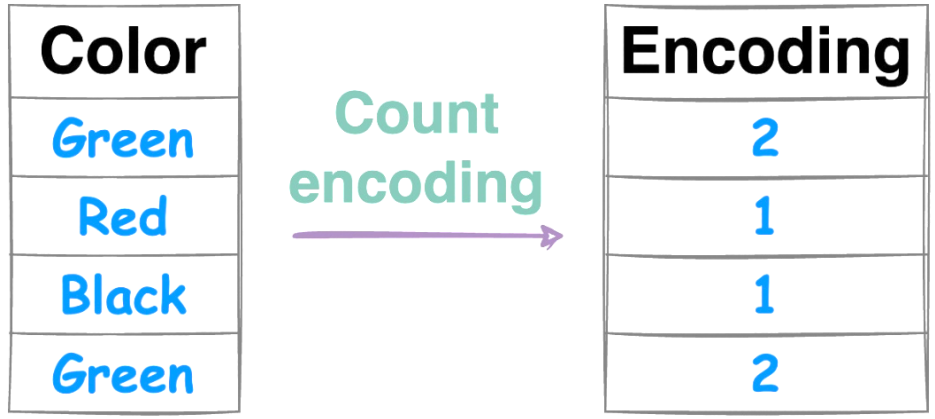
Watch out

Always pass the order. Without it, scikit-learn sorts A–Z: Large=0, Medium=1, Small=2

Avoid for unordered categories with linear or distance-based models: they treat 0 < 1 < 2 as real order

Count / frequency encoding

- Replaces each category with its **count** or **frequency**
  - Does not expand the feature space
- **When to use:** high-cardinality columns (e.g., city, ZIP code) and tree-based models




```python
# Count how often each city appears
counts = df["city"].value_counts()
df["city_count"] = df["city"].map(counts)
# For a share instead: value_counts(normalize=True)


# In a real project: learn counts on TRAIN only
counts = X_train["city"].value_counts()
X_train["city_count"] = X_train["city"].map(counts)
X_test["city_count"] = X_test["city"].map(counts).fillna(0)
# unseen test cities become 0
```

Watch out

Different categories can get the same value (information is lost)

Not suitable for linear models

New categories in test data have no count: fill with 0